

ASSIGNMENTS

과제3 : 실습문제 풀이 7 – 6 / 9 - 4



과 목 : 객체지향언어1[7]

담당교수 : 교수 황기태

학 과 : 빅데이터트랙

학 번 : 2371209

이 름 : 김주혁

일 자 : 2026년 5월 19일 토요일

7 - 6 소스코드

light.h

```
1. #ifndef LIGHT_H // light.h 가 정의된적 없다면 포함
2. #define LIGHT_H // light.h 정의 시작
3.
4. class Light { // Light 클래스 선언부
5.     int brightness; // 빛의 크기를 나타내는 변수 선언
6. public: // 클래스 외부에서 접근 허용
7.     Light(int brightness = 1) { this->brightness = brightness; } // 밝기를 매개변수로 받아 밝기를 저장하는 생성자
8.     Light& operator += (Light& op2); // Light 객체를 매개변수로 하는 += 연산자 오버로딩
9.     Light& operator += (const int op2); // 정수값을 매개변수로 하는 += 연산자 오버로딩
10.    void operator ~ () const; // 빛의 크기를 출력하는 ~ 연산자 오버로딩
11.    Light& operator = (const Light& op2); // Light 객체를 매개변수로 하는 = 오버로딩
12.    Light operator -- (int x); // -- 후위 연산자 오버로딩
13.    bool operator == (const int op2) const; // op2 와 빛의 크기를 비교하는 bool 타입 == 연산자 오버로딩
14. };
15.
16. #endif // light.h 의 끝 정의
17.
```

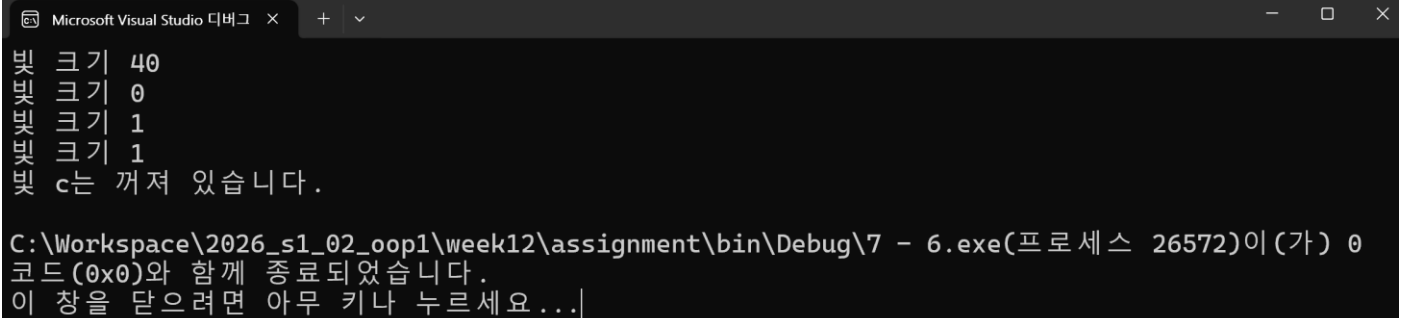
light.cpp

```
1. #include <iostream> // 표준 입출력 라이브러리 가져오기
2. #include "light.h" // light.h 헤더파일 가져오기
3. using namespace std; // std 이름공간 선언 생략
4.
5. Light& Light::operator += (Light& op2) { // Light 객체 op2 를 매개변수로 하는 += 연산자 오버로딩
6.     brightness += op2.brightness; // 자신의 빛의 크기와 op2 의 빛의 크기 더하기
7.     op2.brightness = 0; // op2 의 밝기는 0 이 됨
8.     return *this; // 자기자신 리턴
9. }
10.
11. Light& Light::operator += (const int op2) { // 정수값 op2 를 매개변수로 하는 += 연산자 오버로딩
12.     brightness += op2; // 자신의 빛의 크기와 op2 의 값 더하기
13.     return *this; // 자기자신 리턴
14. }
15.
16. void Light::operator ~ () const { // 자신의 빛의 크기를 출력하는 ~ 연산자 오버로딩
17.     cout << "빛 크기 " << brightness << endl; // 자신의 빛의 크기 출력
18. }
19.
20. Light& Light::operator = (const Light& op2) { // 자신의 빛의 크기에 Light 객체 op2 의 크기를 대입하는 = 연산자 오버로딩
21.     brightness = op2.brightness; // 자신의 빛의 크기는 op2 의 빛의 크기가 된다
22.     return *this; // 자기자신 리턴
23. }
24.
25. Light Light::operator -- (int x) { // 자신의 값을 1 낮추고 낮추기 전 자신의 복사본 리턴하는 -- 후위 연산자 오버로딩
26.     Light tmp = *this; //복사본은 자신의 빛의 크기와 같다
27.     brightness--; //자기자신의 빛의 크기 1 감소
28.     return tmp; // 복사본 리턴
29. }
30.
31. bool Light::operator == (const int op2) const { // 자신과 Light 객체 op2 의 빛의 크기를 비교하는 == 연산자 오버로딩
32.     if (brightness == op2) return true; // 빛의 크기가 같으면 true 리턴
33.     else return false; // 빛의 크기가 서로 다르면 false 리턴
34. }
35.
```

main.cpp

```
1. #include <iostream> // 표준 입출력 라이브러리 가져오기
2. #include "light.h" // light.h 헤더파일 가져오기
3. using namespace std; // std 이름공간 선언 생략
4.
5. int main() { // 메인 함수
6.     Light a(10), b(20), c, d(0); // c는 1, d의 크기는 0
7.     (a += b) += 10; // 빛 a에 b를 더하면 b는 크기를 잃고 0이 된다.
8.     ~a; ~b; ~c; // 빛 a, b, c의 크기 출력
9.     d = c--; // d의 빛 크기는 1이 되고 c의 빛 크기 1 감소, 0이 됨
10.    ~d; // 빛 d의 크기 출력
11.    if (c == 0) cout << "빛 c는 꺼져 있습니다." << endl; // c의 빛의 크기가 0이면 꺼져있음 출력
12.    if (d == 0) cout << "빛 d는 꺼져 있습니다." << endl; // d의 빛의 크기가 0이면 꺼져있음 출력
13. }
14.
```

7 - 6 실행 결과



Microsoft Visual Studio 디버그 콘솔의 실행 결과입니다. 콘솔에는 다음과 같은 출력이 표시되었습니다:

```
빛 크기 40
빛 크기 0
빛 크기 1
빛 크기 1
빛 c는 꺼져 있습니다.

C:\Workspace\2026_s1_02_oop1\week12\assignment\bin\Debug\7 - 6.exe(프로세스 26572)이(가) 0
코드(0x0)와 함께 종료되었습니다.
이 창을 닫으려면 아무 키나 누르세요...
```

9 – 4 소스코드

abstractpoint.h

```
1. #ifndef ABSTRACTPOINT_H // ABSTRACTPOINT_H 가 정의된적 없다면 포함
2. #define ABSTRACTPOINT_H // ABSTRACTPOINT_H 정의 시작
3.
4. #include <string> // 스트링 클래스 가져오기
5. using namespace std; // std 이름공간 선언 생략
6.
7. class AbstractPoint { // AbstractPoint 클래스 선언부
8. protected: // 상속 관계에서 접근 가능한 멤버 선언
9.     string name; // 이름값 저장하는 변수
10.    int point = 0; // 포인트 값을 저장하는 변수
11.    int ratio; // 포인트 적립률을 지정하는 변수 (%)
12. public: // 클래스 외부에서 접근 가능한 멤버 선언
13.     AbstractPoint(string name, int ratio = 1); // 이름과 적립률을 받는 생성자 함수
14.     virtual void earn(int pay) = 0; // 가상 함수
15.     virtual void show() = 0; // 고객의 이름과 포인트를 출력하는 함수
16.     int getPoint(); // 포인트를 리턴하는 함수
17. };
18.
19. #endif // ABSTRACTPOINT_H 정의 끝
20.
```

abstactpoint.cpp

```
1. #include <iostream> // 표준 입출력 라이브러리 가져오기
2. #include <string> // string 클래스 가져오기
3. #include "abstractpoint.h" // abstractpoint.h 헤더파일 가져오기
4.
5. AbstractPoint::AbstractPoint(string name, int ratio) {
6.     // 이름과 적립률을 받아서 초기화 하는 생성자 함수
7.     this->name = name; // 이름 초기화
8.     this->ratio = ratio; // 적립률 초기화
9. }
10.
11. int AbstractPoint::getPoint() { // private 인 point 를 리턴 하는 함수
12.     return point; // point 리턴
13. }
14.
```

9 - 4 소스코드

platinum.h

```
1. #ifndef PLATINUM_H // 중복 방지 선언
2. #define PLATINUM_H // PLATINUM_H 정의 시작
3.
4. #include "abstractpoint.h" // 상속 받을 클래스의 헤더파일 가져오기
5.
6. class Platinum : public AbstractPoint { // AbstractPoint 를 public 으로 상속받는 Platinum 함수 선언부
7. private: // 클래스 외부에서 접근 불가능 멤버 선언
8.     double bonusRatio; // 추가 적립률을 저장하는 변수 선언 (%)
9.     int standardAmount; // 추가 적립률의 기준 금액을 저장하는 변수 선언
10. public: // 클래스 외부에서 접근 가능한 멤버 선언
11.     Platinum(string name, int ratio, int standardAmount, double bonusRatio);
12.     // 고객의 이름, 적립률, 추가 적립률 기준금액, 추가 적립률을 매개변수로 받아 저장하는 생성자 함수
13.     void earn(int pay); // 상속받은 earn 함수 오버라이딩
14.     void show(); // 상속받은 void 함수 오버라이딩
15.     void combine(Platinum& other); // other 의 모든 포인트를 자신의 객체에 넘겨주는 함수 (other 의 포인트는 0 이됨)
16.     void resetPoint(); // 포인트를 0 으로 만드는 함수
17. };
18.
19. #endif // PLATINUM_H 정의 끝
20.
```

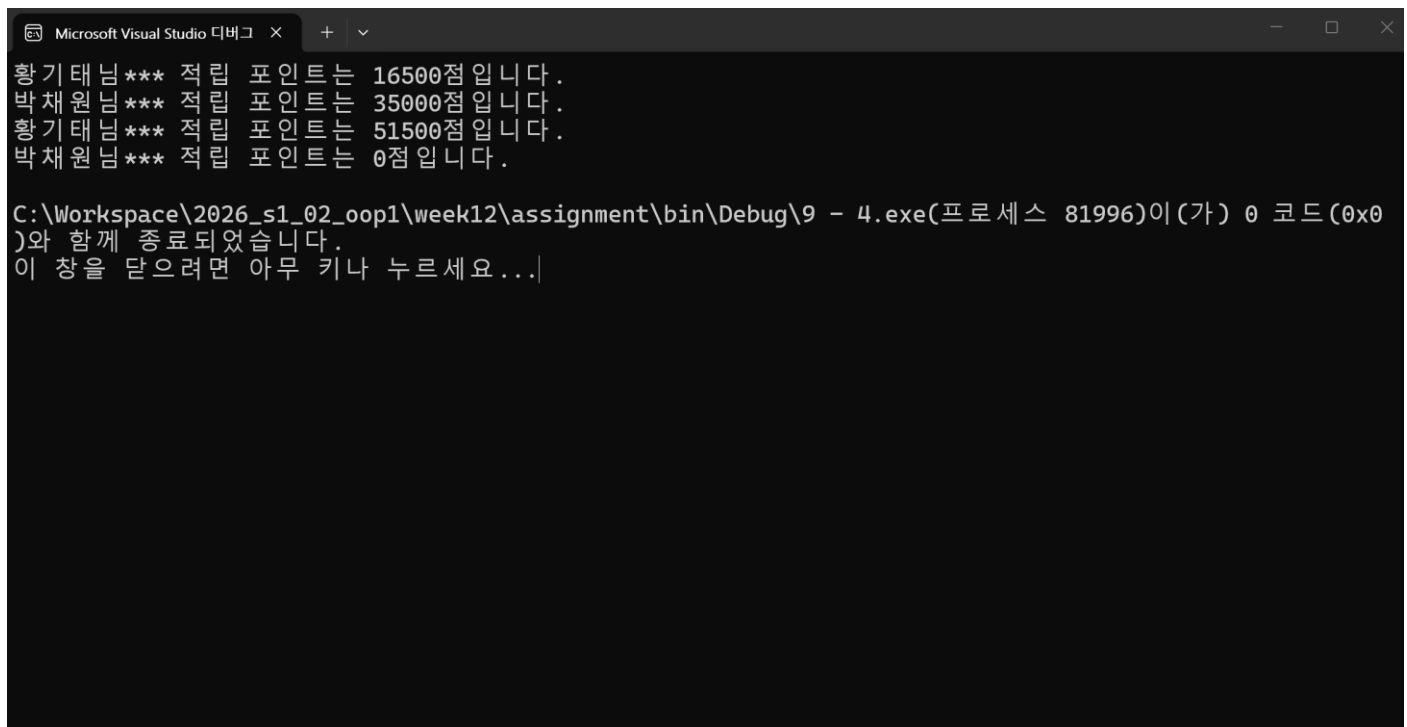
platinum.cpp

```
1. #include <iostream> // 표준 입출력 라이브러리 가져오기
2. #include <string> // string 클래스 가져오기
3. #include "abstractpoint.h" // abstractpoint.h 헤더파일 가져오기
4. #include "platinum.h" // platinum.h 헤더파일 가져오기
5. using namespace std; // std 이름공간 선언 생략
6.
7. Platinum::Platinum(string name, int ratio, int standardAmount, double bonusRatio) : AbstractPoint(name, ratio) {
8.     // 부모 생성자에게 이름과 적립률을 넘겨준뒤 실행되는 Platinum 생성자 함수
9.     this->standardAmount = standardAmount; // 추가 적립률 기준 금액 초기화
10.    this->bonusRatio = bonusRatio; // 추가 적립률 초기화
11. }
12.
13. void Platinum::earn(int pay) { // pay 와 적립률을 이용해 포인트를 누적하는 오버라이딩 함수
14.     if (pay > standardAmount) point += pay * bonusRatio / 100; // 기준 금액 초과면 추가 적립률로 포인트 적립
15.     else point += pay * ratio / 100; // 기준 금액 이하면 기존 적립률로 포인트 적립
16. }
17.
18. void Platinum::show() { // 고객의 이름과 포인트 출력하는 오버라이딩 함수
19.     cout << name << "님*** 적립 포인트는 " << point << "점입니다." << endl; // 이름과 포인트 출력
20. }
21.
22. void Platinum::combine(Platinum& other) { // other 의 포인트를 결합하는 함수
23.     point += other.getPoint(); //other 의 포인트 값 더하기
24.     other.resetPoint(); // other 의 포인트 0 으로 초기화
25. }
26.
27. void Platinum::resetPoint() { // 포인트를 0 으로 초기화 하는 함수
28.     point = 0; // 포인트를 0 으로 초기화
29. }
30.
```

main.cpp

```
1. #include <iostream> // 표준 입출력 라이브러리 가져오기
2. #include <string> // string 클래스 가져오기
3. #include "abstractpoint.h" // abstractpoint.h 헤더파일 가져오기
4. #include "platinum.h" // platinum.h 헤더파일 가져오기
5.
6. int main() {
7.     Platinum hwang("황기태", 5, 100000, 7), park("박채원", 5, 100000, 7);
8.     // 생성자 의미 : 100,000 원까지는 5%, 그 보다 큰 경우 7%
9.
10.    hwang.earn(50000); // 50,000 원 지출 경우, 2500 포인트 적립
11.    hwang.earn(200000); // 200,000 원 지출 경우, 14000 포인트 적립
12.    park.earn(500000); // 500,000 원 지출 경우, 3500 포인트 적립
13.    hwang.show(); park.show(); // 누적 포인트 출력
14.
15.    hwang.combine(park); // hwang 회원에 park 의 포인트 결합
16.    hwang.show(); park.show(); // 현재 누적 포인트 출력
17. }
18.
```

실행 화면

The image shows a screenshot of the Microsoft Visual Studio Debug Console window. The window title is "Microsoft Visual Studio 디버그" with standard window controls. The console output is as follows:
황기태 님*** 적립 포인트는 16500점입니다.
박채원 님*** 적립 포인트는 35000점입니다.
황기태 님*** 적립 포인트는 51500점입니다.
박채원 님*** 적립 포인트는 0점입니다.

C:\Workspace\2026_s1_02_oop1\week12\assignment\bin\Debug\9 - 4.exe(프로세스 81996)이(가) 0 코드(0x0)와 함께 종료되었습니다.
이 창을 닫으려면 아무 키나 누르세요...|

과제 후기

과제를 완성하는데 7 - 9는 40분, 9 - 4는 60분 정도 걸려서 총 약 100분 정도 걸렸습니다.

실습 문제 7 - 6은 빛을 추상화한 클래스 Light의 연산자 오버로딩을 하는 문제입니다. 이 문제를 풀 때 중요하게 생각한 것은 const와 &를 상황에 맞게 쓰는 것이었습니다. 연산자 +=는 $a += b += c$ 같이 체이닝이 가능하게끔 반환 값을 참조 리턴으로 하였고 매개 변수는 Light 객체일때는 값이 더해진 후, op2의 빛의 크기가 0이 되어야 하기 때문에 const를 사용하지 않고 참조로 받게 하였습니다. 매개 변수가 int형일 때는 op2의 값이 바뀌지 않는다는 의미로 const를 붙였습니다. ~ 연산자는 자기 빛의 크기를 출력하는 함수로 전위 연산자이기에 매개변수를 주지 않았습니다. = 연산자는 += 연산자와 마찬가지로 중복 호출이 가능하도록 반환값을 참조 리턴으로 하였고 매개변수의 값을 바꿀 수 없도록 const로 하였습니다. - 연산자는 후위 연산자이기 때문에 매개변수를 int x로 하였고 복사본을 리턴하게 하였습니다. == 연산자는 두 값 모두 비교만 하고 값을 변경하지는 않기 때문에 자신과 매개변수 모두 const로 하였습니다.

연산자 오버로딩을 하였기 때문에 연산자들이 기본적인 연산의 기능을 하는 것을 넘어서 Light 객체를 사용하였을 때도 프로그래머가 의도 한 대로 작동하게 됩니다. 이는 객체지향언어의 다형성, 함수 중복의 특징을 잘 나타내는 방식입니다.

실습 문제 9 - 4는 AbstractPoint 클래스를 상속받아 플래티넘 회원의 포인트 적립을 관리하는 Platinum 클래스를 작성하는 문제입니다. Platinum 클래스는 높은 회원 등급이라는 개념이기에 기존 금액이 넘는 지출을 했을 때 기존 적립률이 아닌 더 높은 적립률로 적립을 받는 클래스입니다. 때문에 기존 멤버인 이름 변수 name과 적립률 ratio 변수뿐만 아니라 초과 적립률을 받기 위한 기준 금액인 standardAmount 변수와 초과 적립률을 저장하는 멤버 변수를 추가로 만들었습니다. 기존에 있는 AbstractPoint 클래스에서 earn()함수와 show()가 가상 함수로 작성 되었기 때문에 platinum 클래스에서 오버라이딩 해주었습니다. 이렇게 되면 만약 AbstractPoint* p = new Platinum("황기태", 5, 100000, 7); p->earn(50000); 라고 해도 동적 바인딩이 일어나 AbstractPoint 클래스의 earn이 호출되는 것이 아니라 Platinum의 earn가 실행되게 됩니다. combine 함수에서는 point가 protected로 선언되어 있어서 other.point로 직접 접근이 가능하지만, 캡슐화의 원칙이 깨지기 때문에 getPoint()와 resetPoint()로 경유하여 접근하도록 하였습니다. 메인함수에서 Platinum 객체를 생성하면 먼저 부모 클래스인 AbstractPoint의 생성자가 name과 ratio를 받아서 실행 된 뒤 자식 클래스인 Platinum 생성자가 실행 돼 standardAmount와 bonusRatio를 초기화합니다. Earn 함수와 show 함수가 부모 클래스에서 가상 함수로 선언 되어있기 때문에 만약 Platinum 객체가 AbstractPoint로 업캐스팅 되어 있다고 하더라도 동적 바인딩이 일어나 런타임 도중에 실제 객체인 Platinum을 보고 Platinum의 멤버 함수를 실행합니다.

이 문제는 다른 사람이 만든 클래스 가져올 때 어떤 생각을 해야 하는지 가르침을 주는 문제 같습니다. AbstractPoint 클래스에서 earn과 show()를 가상 함수로 만들어 놓았기 때문에 다른 사람이 이 코드를 재사용할 때 작동 방식을 유추할 수 있게 되고原作者의 의도를 알 수 있게 됩니다. 이 문제만 놓고 본다면 동적 바인딩의 의미가 크게 있진 않지만, 만약 silver나 gold같은 클래스를 추가로 만든다고 할 때 각각의 등급에 따라 동적 바인딩이 일어나 그 함수를 따라가기 때문에 효율성이 커지게 됩니다.

이번 과제에서는 각각 함수의 오버로딩과 오버라이딩을 구현하는 문제를 풀어봤습니다. 이번 과제를 통해서 오버로딩과 오버라이딩의 차이를 명확하게 구분할 수 있게 되었고 가상 함수의 의의를 깨닫게 되었습니다.